

Task Management REST API

This documentation provides complete implementation details, architecture design, authentication flow, validation rules, security mechanisms, API endpoints, testing strategy for Task Management REST API.

1. Overview

This project is a RESTful Task Management API built using:

- Java 21
- Spring Boot
- Spring Security
- JWT Authentication
- Spring Web MVC
- Spring Data JPA
- Hibernate
- Hibernate Validator
- H2 [In-Memory Database]
- Spring Actuator [for metrics and observability]
- Junit
- Mockito.

The system allows users to: Register, Login

and authenticated users to: Logout, Create Tasks, Retrieve Tasks, Update Tasks, Delete Tasks, Mark Tasks as Completed, Change password.

Additional Features:

- JWT Authentication,
- Token Blacklist for Logout Mechanism
- Pagination Support
- API Versioning
- Global Exception Handling
- Data Validation

- Optimistic Locking
- Integration Testing
- Unit Testing
- Spring Actuator for Observability
- Swagger/OpenAPI Documentation

2. API Documentation Tools

Tool	Purpose
Swagger / OpenAPI	Interactive HTML for testing endpoint
SpringDoc OpenAPI	OpenAPI v3 specification autogenerated documentation
Postman Collections	User intended endpoint with sample data for testing

Swagger UI Endpoint:

```
/swagger-ui/index.html
```

OpenAPI Specification:

```
/v3/api-docs
```

3. Architecture

The application follows layered architecture for separation of concerns and maintainability.

N-tier Architecture Flow

Controller Layer → Service Layer → Repository Layer → Database

Package	Responsibility
controller	REST endpoints
service	Business logic
repository	Database access
entity	JPA entities and DTOs
configuration	JWT Filter and Spring Security configuration
exception	Global exception handling and Custom Exception
test	Unit test and Integration tests

4. Authentication Flow

Registration

POST /auth/register

Passwords are hashed using BCrypt before storage.

Login

POST /auth/login

Valid credentials generate JWT tokens signed using HMAC SHA 256 and are returned to clients.

Authorization

Every request must have below header

Authorization: Bearer <token>

JWT filter: Extracts token ,Validates it, Authenticates request or Reject blacklisted tokens

Logout

POST /auth/logout

JWT token is stored in blacklist until expiration.

5. Task Entity

Field	Type	Description
id	long	Unique task ID
title	String	Task title
description	String	Task description
dueDate	LocalDate	Task due date
status	Enum	PENDING / IN_PROGRESS / COMPLETED
createdAt	Instant	Task creation timestamp
updatedAt	Instant	Last update timestamp
version	long	Optimistic locking version

6. API Endpoints

Method	Endpoint	Description
POST	/auth/register	Register user
POST	/auth/login	Authenticate user
POST	/auth/logout	Logout
POST	/api/v1/tasks	Create task
GET	/api/v1/tasks	Retrieve all tasks
GET	/api/v1/tasks/{id}	Retrieve task by ID
PUT	/api/v1/tasks/{id}	Update task
DELETE	/api/v1/tasks/{id}	Delete task
PATCH	/api/v1/tasks/{id}/complete	Mark task as complete

Sample Create Task Request

```
{
  "title": "Implement Feature",
  "description": "Complete API integration",
  "dueDate": "2026-06-20",

  "status": "IN_PROGRESS"
}
```

Sample Response

```
{ "id": 1,

  "title": "Implement Feature",

  "description": "Complete API integration",

  "dueDate": "2026-06-20",

  "status": "IN_PROGRESS",

  "createdAt": "2026-05-13T10:00:00Z",

  "updatedAt": null,

  "version": 0 }
```

7. Validation Rules

User Validation

Email must be valid.

Password must be minimum 8 characters and contain 1 uppercase letter,1 number,1 special character.

Task Validation and DB Constraint

Title must not be blank.

Due date must be future or present.

Created At,Updated At must be Past or Present

ID must be positive

Status must not be null

8. Error Handling

The application uses centralized exception handling using:

@RestControllerAdvice

Errors are returned using structured ProblemDetail responses.

```
{
  "title": "Invalid Task",
  "status": 404,
  "detail": "Task doesn't exist with id : 10",
  "traceId": "uuid-value"
}
```

9. Security Features

JWT Authentication with Stateless session where password are Hashed using BCrypt before storing. Token Blacklist for Logout functionality.

10. Optimistic Locking

The Task entity uses:

```
@Version private long version;
```

Purpose: Prevents lost updates, Handles concurrent modifications safely.

11. Testing

Unit Testing

JUnit 5 and Mockito are used to assert behaviour when Duplicate user validation, Task retrieval.

Integration Testing

SpringBootTest MockMvc are used where Registration of user ,Login ,Task creation, Task retrieval, Logout are done.

12. Running the Application

Prerequisites:

- Java 21+ • Maven

Clone Repository

```
git clone cd Wellness360
```

Run Application

```
mvn spring-boot:run
```

Run Tests

```
mvn test
```

13. Design Decisions

JWT Authentication :

For Stateless authentication which is Scalable and REST API friendly

Optimistic Locking:

Prevents concurrent update issues. Supports multi-user environments

DTO:

Protect internal entity structure, Prevent accidental primary key modification, Ensure valid payload structures

Global Exception Handling:

Consistent error responses and Centralized logging for Cleaner controllers and Easier debugging

14. Future Improvements

Refresh token can be implemented for reauthentication without log in again. User id must be in task entity which is intentionally left to align with given task entity structure which prevent user seeing other users task.

15. Conclusion

This project demonstrates: REST API design, Backend development using Java and Spring Boot, Authentication using JWT, Data Validation, Exception handling, Layered architecture, Testing, Optimistic locking, API versioning, Pagination, Secure password handling.